

A Comprehensive Study of Bugs in Relational DBMS

Shuang Liu, Ruifeng Wang, Yuanfeng Xie, Junjie Chen, Wei Lu, Xiao Zhang, Quanqing Xu, Chuanhui Yang, Xiaoyong Du

Abstract—Relational Database Management Systems (RDBMSs) are crucial infrastructures supporting a wide range of applications, making bug mitigation within these systems essential. This study presents the first comprehensive analysis of bugs in three popular open-source RDBMSs—MySQL, SQLite, and openGauss. We manually examined 777 bugs across four dimensions, i.e., bug root causes, bug symptoms, bug distribution across modules, and the correlations between the studied aspects. We also analyzed the bug-triggering SQL statements to uncover test cases that cannot be generated by existing tools. We have made 12 findings, which throw lights on the development, maintenance and testing of RDBMS systems. Particularly, our findings reveal that bugs related to SQL data types and complex features, such as database triggers, procedures and database parameter settings, present significant opportunities for enhancing RDBMS bug detection and mitigation. Leveraging these insights, we developed a tool, SQLT, which effectively identified eight RDBMS bugs (five type-related), all verified by developers, with four subsequently fixed.

Index Terms—Empirical Study, Relational Database, Bugs

1 INTRODUCTION

Relational Database Management Systems (RDBMSs) [1], [2], [3], [4] are fundamental infrastructures for a wide range of application systems and various business domains, including education [5], healthcare [6], E-commerce [7], technology software [8] and media [9]. They play a pivotal role in handling large volumes of structured data efficiently, maintaining transaction integrity, and supporting complex queries. This functionality underpins operational effectiveness and informed decision-making in these sectors [10].

There is a critical bug in SQLite that causes a segmentation fault due to improper context binding of nested aggregate functions. The bug is triggered by a query with deeply nested window and aggregate expressions (see Figure 1). In the outermost layer ①: A window function `SUM(0) OVER (...)` is used, where the `OVER` clause contains a nontrivial `ORDER BY` expression; In the `ORDER BY` expression ②, the

```
CREATE TABLE a(b);

①SELECT SUM(0) OVER(
②  ORDER BY
③    (SELECT max(b) OVER (
④      PARTITION BY SUM((
⑤        SELECT b FROM a UNION SELECT b ORDER BY b)))
    INTERSECT
    SELECT b FROM a ORDER BY b))
FROM a;
```

Fig. 1: Motivation Example

sort key is a subquery `SELECT INTERSECT SELECT`; ③: This subquery includes a window function `max(b) OVER (...)`; ④: Within the inner `OVER`, the `PARTITION BY` expression contains a call to `SUM(...)`, i.e., the second-level aggregation; ⑤: The `SUM()` is applied over a subquery combining `UNION` and `ORDER BY`.

The root cause lies in how SQLite internally tracks aggregation usage via the `AggInfo` structure. In this query, the inner `SUM()` (④) is embedded inside multiple layers of window and scalar subqueries. SQLite fails to register this aggregation in the correct context, resulting in an uninitialized access duaring query execution. This ultimately causes a segmentation fault and crashes the system, since the planner fails to detect and organize the aggregate function within the appropriate aggregation level. This bug is difficult to expose through conventional SQL testing tools due to its complex feature combinations and deep levels of nested SQL queries.

Given the critical role that RDBMSs play in various systems, it is crucial to effectively detect and address bugs in these databases. There have been various testing approaches [11], [12], [13], [14], [15], [16], [17], [18], [19], [20], [21] aiming at detecting bugs in RDBMSs, and they successfully detected new bugs in RDBMSs. These approaches generate large numbers of diverse SQL statements through random test case generation or mutation, aiming to expose

- Shuang Liu is with School of Information, Renmin University of China, China. E-mail: shuang.liu@ruc.edu.cn
- Ruifeng Wang is with the College of Intelligence and Computing, Tianjin University, China. E-mail: ruifeng_wang@tju.edu.cn
- Yuanfeng Xie is with the College of Intelligence and Computing, Tianjin University, China. E-mail: yuanfeng_xie@tju.edu.cn
- Junjie Chen is with the College of Intelligence and Computing, Tianjin University, China. E-mail: junjiechen@tju.edu.cn
- Wei Lu is with Key Laboratory of Data Engineering and Knowledge Engineering (MOE), Renmin University of China, China. E-mail: luwei@ruc.edu.cn
- Xiao Zhang is with School of Information, Renmin University of China, China. E-mail: zhangxiao@ruc.edu.cn
- Quanqing Xu is with OceanBase, Ant Group. E-mail: xuquanqing.xqq@oceanbase.com
- Chuanhui Yang is with OceanBase, Ant Group. E-mail: rizhao.ych@oceanbase.com
- Xiaoyong Du is with Key Laboratory of Data Engineering and Knowledge Engineering (MOE), School of Information, Renmin University of China, China. E-mail: duyong@ruc.edu.cn

Corresponding author: Xiao Zhang.

TABLE 1: Comparison with Prior Empirical Studies on RDBMS Bugs

study	bug type	duration	# bugs
Fonseca [22]	concurrency bugs	2003-2009	80
Cui [23]	transaction bugs	2018-2022	140
ours	general bugs	2021-2023	777

potential DBMS bugs. However, they typically support only a limited subset of SQL features and do not target specific components of RDBMSs. As a result, they are unable to detect bugs that arise from complex SQL functionalities, such as database events and triggers, stored procedures, access control mechanisms, or specific database parameter configurations. These bugs count for 33.72% of our studied bugs (See section 3.6 for detailed analysis). “ Effectively detecting RDBMS bugs remains challenging, and one important gap is the lack of general and deep understanding of RDBMS bugs, especially functional bugs.

Another research direction involves empirical studies on RDBMS bugs, focusing on specific issues like concurrency bugs [22] and transaction bugs [23], to gain a deeper understanding of these problems. Fonseca et al. [22] analyzed 80 concurrency bugs over six years, exploring their impacts and distribution. Similarly, Cui et al. [23] examined 140 transaction bugs over five years, offering a detailed classification from a transactional perspective. While both studies provide thorough analysis of specific bug types in RDBMSs, their scope—regarding the number and types of bugs studied—is limited. Among the bugs we studied, only 1.29% are related to transactions and 5.02% to concurrency, leaving the remaining 93% of bugs unexplored in prior work. Currently, there is no comprehensive empirical study covering general RDBMS bugs in the literature.

To bridge the gap in existing research works, we conducted the first large-scale, comprehensive study of bugs in RDBMSs, selecting three representative systems: MySQL [1], SQLite [2], and openGauss [4]. Table 1 compares our study with previous studies on RDBMS bugs. We analyzed 777 bugs from these systems, focusing on their root causes, symptoms, the modules where fixes occur, and the interrelations between those aspects. From our extensive analysis, we identified 12 key findings, which provide insights for RDBMS development, testing and maintenance. Notably, bugs related to SQL types and transactions are often overlooked. Inspired by these findings, we developed a straightforward method and a proof-of-concept tool, SQLT, to detect SQL-type related bugs and improve RDBMS testing. SQLT successfully identified eight unique bugs across the three studied RDBMSs, all confirmed by developers, with seven reported for the first time and five subsequently fixed. These results underscore the practical value and potential of our research in guiding the design of more effective testing tools for RDBMSs.

To sum up, we make the following contributions.

- We conducted the first comprehensive study of RDBMS bugs, based on a dataset of 777 RDBMS bugs collected from three popular RDBMSs: MySQL, SQLite, and openGauss. Our study delved into the root causes of the bugs, the symptoms they manifest, and the specific RDBMS modules where the fixes are

conducted, providing a holistic view on the bugs that affecting these critical systems.

- Our study yields 12 insightful findings, each contributing valuable guidelines for future RDBMS development, testing and maintenance.
- Based on our findings, we propose a preliminary testing method SQLT, which successfully detected 8 bugs across three major RDBMS platforms—2 in MySQL, 1 in SQLite, and 5 in openGauss—demonstrating the practical applicability of our findings. We’ve made our dataset ¹ and tool ² open source to spur future research.

2 METHODOLOGY AND CLASSIFICATION

2.1 Research Questions

By conducting the empirical study, we would like to answer the following research questions.

RQ1: What are the common root causes of the bugs in relational database management systems? Identifying the root causes of bugs is crucial, as it highlights poor programming practices and flawed design decisions. This understanding not only informs the development of more effective testing methods to detect such issues, but also helps prevent the recurrence of similar bugs by encouraging better coding habits and design choices in the future.

RQ2: What are the symptoms of the bugs? Recognizing the symptoms of bugs in an early stage is important for software developers, as it not only streamlines the debugging process but also equips programmers with the insight to prevent the introduction of similar bugs, which enhances the overall software quality and development efficiency.

RQ3: Where in the system are the bugs fixed? Understanding which software components contain bugs is vital, especially in collaborative development environments where different teams are responsible for various components or functionalities. This information can also reflect the level of testing effort allocated to each component, indicating where additional testing may be necessary. It helps in reallocating resources efficiently to ensure thorough examination and reliability of software components.

RQ4: Are there correlations between the measured aspects? Analyzing the correlations between root causes and symptoms offers valuable insights into preventing similar issues during future development process, and guides more targeted and effective testing and debugging strategies.

2.2 Databases to be Studied

In the study, we choose three popular open-source RDBMSs, including MySQL[1], SQLite[2], and openGauss[4], as subjects for bug collection and study. We first consulted the DB-Engines Ranking [24] to identify ranked and widely adopted RDBMS, and surveyed existing testing approaches [11], [12], [14], [19], [13], [15], [16], [17], [18], [20], [21], [25], [26] as well as empirical studies on RDBMS bugs [27], [23]. This led us to identify potential candidates such as MySQL, SQLite, PostgreSQL, TiDB, MariaDB, DuckDB, and openGauss. After further evaluation, we selected three representative

1. <https://github.com/sqlttest/studydataset>

2. <https://github.com/sqlttest/SQLT>

RDBMSs—MySQL, SQLite, and openGauss—based on two criteria: design architecture and application scenarios.

We categorized the candidate RDBMSs into three groups based on these criteria: (1) MariaDB and TiDB, which are designed to be highly compatible with MySQL on system architecture and SQL syntax, making them well-suited for small to medium-sized systems and distributed databases; (2) SQLite and DuckDB, which are embedded databases ideal for embedded systems and lightweight applications, with SQLite being more widely adopted and popular [24]; (3) openGauss and PostgreSQL are high-performance RDBMSs tailored for large-scale and complex systems. openGauss shares many architectural similarities with PostgreSQL but offers additional features and optimizations. We select one representative, actively maintained RDBMS from each group, covering different architectures and application scenarios.

MySQL is a full-featured RDBMS known for its robustness and versatility, supported by Oracle and with a large community and ecosystems. It uses a client-server model and is typically hosted on a server, accessible to multiple clients over a network. This design and architecture make it suitable for web applications, e-commerce platforms, and as a back-end database for enterprise applications. MySQL has good scalability and cross-platform capabilities.

SQLite is distinguished by its lightweight, ubiquitous, and embedded characteristics, making it suitable for resource-constrained environments, such as embedded systems, small to medium websites, and applications requiring a portable database without the overhead of a server setup. Unlike MySQL, SQLite is serverless, embedded into the end programs and runs in-process with the application, making it ideal for lightweight applications like mobile apps or small desktop applications. The lightweight design also makes it not suitable for high-concurrency environments or very large databases.

openGauss is an enterprise-grade, open-source relational database focused on high-performance and high-reliability scenarios, often used in cloud environments. It is designed to support large-scale, high-concurrency operations and offers features like partitioning and parallel query execution. The core of openGauss is derived from PostgreSQL [3], and is deeply optimized for ARM architecture chips and is compatible with the X86 architecture, making it suitable for enterprise applications.

To summarize, the three databases differ significantly in terms of design and architecture, scalability and application scenarios. The bugs found in these RDBMSs are representative of the challenges faced across various database systems.

2.3 Bug Collection

Following existing empirical studies on software bugs [28], [29], [30], [31], we collected bugs of the studied databases from their official bug repositories, i.e., the SQLite bug report website [32], MySQL commits in MySQL GitHub repositories [33], and the openGauss commits in Gitee repositories [34]. We gathered bugs from the corresponding repositories based on three key criteria. First, we concentrated on bugs within the core functionalities of the RDBMS, such as SQL parsing, execution, optimization, and data storage,

TABLE 2: Statistical Information on Collected Bugs

RDBMS	Rank(DB-Engine)	LOC(K)	Duration	Total	Analyzed
MySQL	2	4470	2021.9-2023.4	896	231
SQLite	7	392	2018.1-2023.4	335	270
openGauss	87	2200	2021.9-2023.4	1264	276
Total	-	-	-	2495	777

excluding those related to the client and other auxiliary functionalities. Second, the bug report must include detailed descriptions, including information on bug-triggering test cases, the environment, and symptom descriptions. Lastly, the bug must be resolved, with a documented code fix and developers' comments about the fix. These criteria help us more accurately identify the symptoms, root causes, and the origin modules of the bugs, with minimal subjective judgment on the conclusions.

Table 2 presents the statistics of the RDBMS bugs we collected. For MySQL and openGauss, we gathered bugs over a period of one and a half years (September 2021 to April 2023), while for SQLite, we extended the collection period to five years. This adjustment was necessary because only 11 bugs were reported for SQLite between September 2021 and April 2023, insufficient for statistical analysis. Most bugs for SQLite were reported from 2018 to 2021, prompting us to extend the collection period back to January 2018 for SQLite. In total, we collected 2495 bugs, comprising 896 in MySQL, 335 in SQLite, and 1264 in openGauss for the respective periods. After applying the three filtering criteria, 777 bugs remained—231 from MySQL, 270 from SQLite, and 276 from openGauss. The dataset is available in <https://github.com/sqlttest/studydataset>.

2.4 Bug Classification Method

Bug Labelling. For each bug, we assign three labels: root cause, symptom, and fixed module. The symptom is readily identifiable from the bug report. The module where the fix occurred can be determined from the fix records, which include details at both the file and code line levels. Identifying the root causes is more challenging, as it requires a thorough analysis of bug fix details, symptom descriptions, and developer discussions in the repository. We assign the root cause label for each bug based on our manual analysis on this information, supplemented by code inspection when necessary. Since all analyzed bugs have been fixed, the bug fix details often provide valuable insights for pinpointing the root causes.

Achieving Conformance. To ensure objective and accurate bug labeling, two authors independently analyzed and labeled the collected bugs across categories of symptoms, root causes, and fixed modules. They then conducted cross-validation and discussed their assessments until a consensus was reached on the labeling results. If a consensus could not be achieved, a third author was involved in the validation and discussion process until an agreement was reached. We used Cohen's Kappa coefficient [35] to assess label agreement. Thanks to our comprehensive filtering criteria, each analyzed bug was associated with detailed information from bug reports and fixed files, which facilitated the accurate labeling of bug symptoms and fixed modules. Labeling the root causes proved more challenging, with an initial kappa score of 63.82%. After further discussion, the kappa

coefficient increased to 100%, indicating full label agreement among the authors.

Bug Categories. We established the initial classifications for root causes and bug symptoms based on prior bug studies in compilers [28], [29], [30], [36], [37], [23]. The initial classification of RDBMS modules was derived from existing research on RDBMS architectures [10], along with the specific architectures of the RDBMSs we examined. As the labeling process progresses, we continuously refine these classifications. Whenever a bug does not neatly fit into an existing category, or it spans multiple categories, we adjust the labeling system to accommodate it. This iterative process ensures our categorizations remain accurate and relevant throughout the analysis.

Conceptual distinctions and novelty. Previous empirical studies of RDBMS bugs (e.g., [22], [23]) primarily conducted functional-level classifications, for instance, determining whether a transaction violates ACID properties or whether concurrency anomalies occur. Such classifications mainly capture the observable behavior of bugs, but leave their underlying root causes unexplored. In contrast, our taxonomy introduces a complementary four-dimensional bug analysis methodology consisting of root causes, symptoms, modules and the correlations among those aspects:

Root-cause dimension: uncovers the fundamental mechanisms of bug occurrence, including incorrect assertion/exception handling, incorrect storage management, incompatibility, API misuse, and SQL type problems—categories rarely recognized in prior work.

Symptom dimension: describes the externally observable effects of bugs, such as wrong query results, crashes, hangs/deadlocks, and performance regressions, providing immediate clues for testing and diagnosis.

Module dimension: identifies the system component where the bug is ultimately fixed (e.g., parser, optimizer, executor, storage engine, process manager), thereby indicating the internal location and distribution of different bug types.

Correlations between these three dimensions: we connect symptoms with their corresponding root causes, and leverage module information to narrow down the search space, enabling more effective bug localization and deeper analysis. This four-dimensional perspective lays a stronger foundation for root-cause reasoning and testing methodology design, while also allowing us to identify categories and patterns that were previously unexplored.

2.4.1 Bug Root Causes

We have identified and classified the root causes of RDBMS bugs into the following 12 categories.

Incorrect Code Logic: This category encompasses bugs stemming from flawed logic implementations, e.g., errors in code control flow and functionality implementation. These bugs are often difficult to identify and may require extensive code revisions to rectify.

API Misuse: This involves incorrect usage of application programming interfaces (APIs) or libraries by developers. We identified four scenarios for API misuse: (1) missing/redundant API usage, (2) incorrect API argument, (3) incorrect API usage and (4) incorrect API usage timing.

Type Problem: This refers to issues related to incorrect type conversion or misuse of variable types. We have further di-

vided type-related bugs into two subcategories—type problems within SQL and type problems in the programming language (e.g., C/C++) of the RDBMS.

Missing Code Logic refers to the cases of missing logic in a program due to lacking consideration for specific scenarios or missing implementation of functionalities. Different from incorrect code logic, which refers to an existing implemented logic that leads to incorrect or undesired behavior due to errors, missing code logic refers to the absence of necessary logic to handle specific scenarios or conditions.

Variable Management Problem: This includes issues related to improper initialization/modification of variable values, errors in logical/numerical computations, and overflow of variable values.

Incorrect Error/Assertion Handling: This can be divided into two types. The first type involves flaws within the error handling system of the database, such as inappropriate error messages or errors reported at incorrect times. The second type concerns issues with the incorrect assertion conditions.

Incorrect Storage Management: This involves bugs associated with the initialization, allocation, and cleanup of various storage, including cache, page, register, and memory.

Concurrency: These bugs result from incorrect programming in multi-threaded or multi-process environments, including issues like deadlocks and race conditions.

Null Pointer Problem: This refers to bugs caused by accessing an uninitialized pointer or a pointer being assigned a NULL value.

Incompatibility: Bugs in this category arise due to program evolution, such as update to functions and features, leading to compatibility issues. These are further divided into internal incompatibility (issues within an RDBMS) and external incompatibility (between an RDBMS and external libraries).

Misconfiguration: Issues here are due to incorrect configurations within the RDBMS project settings.

Typo: Bugs in this category are due to spelling or typographical mistakes within the RDBMS code.

2.4.2 Bug Symptoms.

The second aspect of our analysis focuses on bug symptoms, which encompass explicit error messages, system status changes (such as hangs), execution results, and execution times. We have identified 7 distinct bug symptoms:

Result Inconsistency: The returned results differ from the expected outcomes based on the query semantics. It may manifest as an inconsistent number of records returned or discrepancies in record values.

Unexpected Error: This refers to situations where the RDBMS reports errors despite the system operating correctly, or when the reported error messages are incorrect.

Assertion Failure: This symptom occurs when the program terminates unexpectedly and displays an assertion failure message, indicating a serious flaw in program logic.

Crash: The RDBMS shutting down or exiting the running services abruptly, sometimes with logs detailing the crash.

SegFault: A segmentation fault typically causes the program to terminate abruptly, often displaying a "Segmentation fault (core dumped)" message in the console or log files.

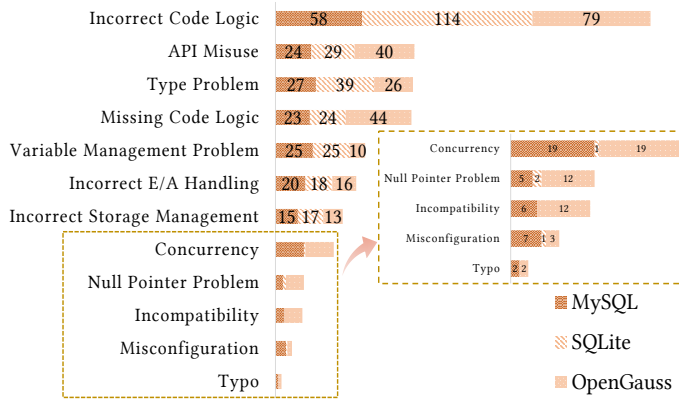


Fig. 2: Bug Distribution by Root Causes

Bad Performance: This symptom relates to efficiency issues, such as delayed response times or poor resource utilization during task execution, resulting in prolonged execution times or excessive resource consumption.

Hang: In RDBMS contexts, a hang describes a scenario where the system becomes unresponsive or suspended during normal operations, unable to process requests or execute queries. Hangs can result from various issues, including deadlocks or the execution of lengthy queries.

2.4.3 Bug Fix Locations

The third aspect we analyzed is the location or module where the bugs are fixed, which helps guide the allocation of resources for testing and maintenance. We developed a general module partition for RDBMS based on existing research on RDBMS architectures [10] and the documentation of the specific RDBMSs we studied [38], [39], [40].

Parser: The parser is crucial for converting SQL query statements into an internal format. It conducts lexical, syntax, and some semantic analyses of SQL statements to ensure their validity.

Optimizer: The optimizer is a complex component for enhancing SQL query performance. It evaluates multiple execution plans to find the most efficient one, considering factors like indexes, table metadata, and query operators.

Executor: The executor is responsible for executing the optimized plan from the Optimizer. It retrieves data, manages transactions, handles errors, effectively transforming an abstract plan to actual results.

Storage Engine: This module manages data storage and retrieval on disk. It organizes physical data storage, manages indexes, and handles data operations like insertion, deletion and querying.

Shared Components and Utils: This module includes auxiliary functions within the RDBMSs, such as replication (MySQL) and catalog management (openGauss), which support both user-facing features and internal operations.

Plugin: Plugins provide optional extension functions, allowing customization and enhancement of the RDBMS to meet specific needs without altering the core codebase.

Process Manager: This module manages concurrent execution. It coordinates various threads, ensuring efficient and synchronized processing of DBMS operations.

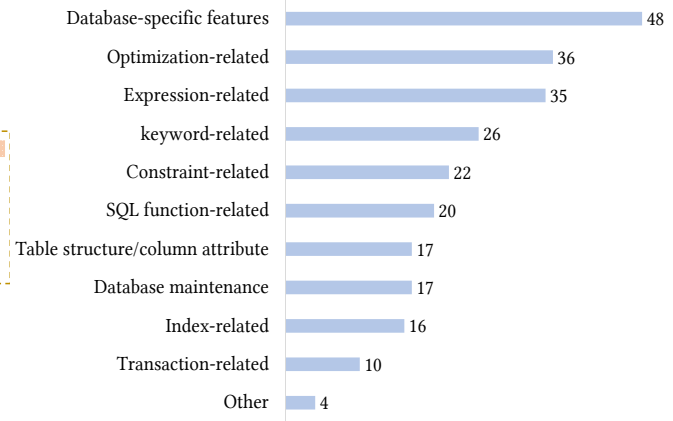


Fig. 3: Code Logic Bug Distributions with Functionality

3 RESULTS AND ANALYSIS

In this section, we present and discuss the experimental results addressing our research questions.

3.1 RQ1: Root Causes

Figure 2 illustrates the distribution of RDBMS bugs according to their root causes. It reveals that incorrect code logic is the predominant root cause, encompassing 251 instances (58 from MySQL, 114 from SQLite, and 79 from openGauss), which represent 32.30 % of all documented bugs.

The primary reason for the prevalence of incorrect code logic bugs stems from the complexity and broad functionality of RDBMSs. These systems offer an extensive array of features, including a vast repertoire of SQL keywords, sophisticated indexing mechanisms to enhance query performance, and a variety of optimization algorithms. Furthermore, they support critical functionalities such as master-slave replication, transaction handling and logging, crucial for maintaining ACID properties. This multifaceted nature significantly increases the likelihood of logical errors during development. To gain a deeper understanding of the bugs related with incorrect code logic, we categorized them based on the functionalities they affect:

- **Optimization-related:** Bugs linked to database optimization functionalities.
- **Constraint-related:** Bugs concerning integrity constraints, such as uniqueness or non-null constraints.
- **Index-related:** Bugs in the construction and usage of database indexes.
- **Keyword-related:** Bugs triggered by using a single or the combination of SQL keywords, such as `JOIN` and `ORDER BY`.
- **Expression-related:** Bugs caused by the manipulation of expressions in SQL queries, such as those involving string, logical, comparison, arithmetic operators, and subqueries.
- **SQL function-related:** Bugs arising from SQL aggregate functions, window functions, and other built-in functions.
- **Database-specific features:** Bugs unique to specific database implementations, like table partitioning in openGauss, full-text search in SQLite, and variable customization in MySQL and openGauss.

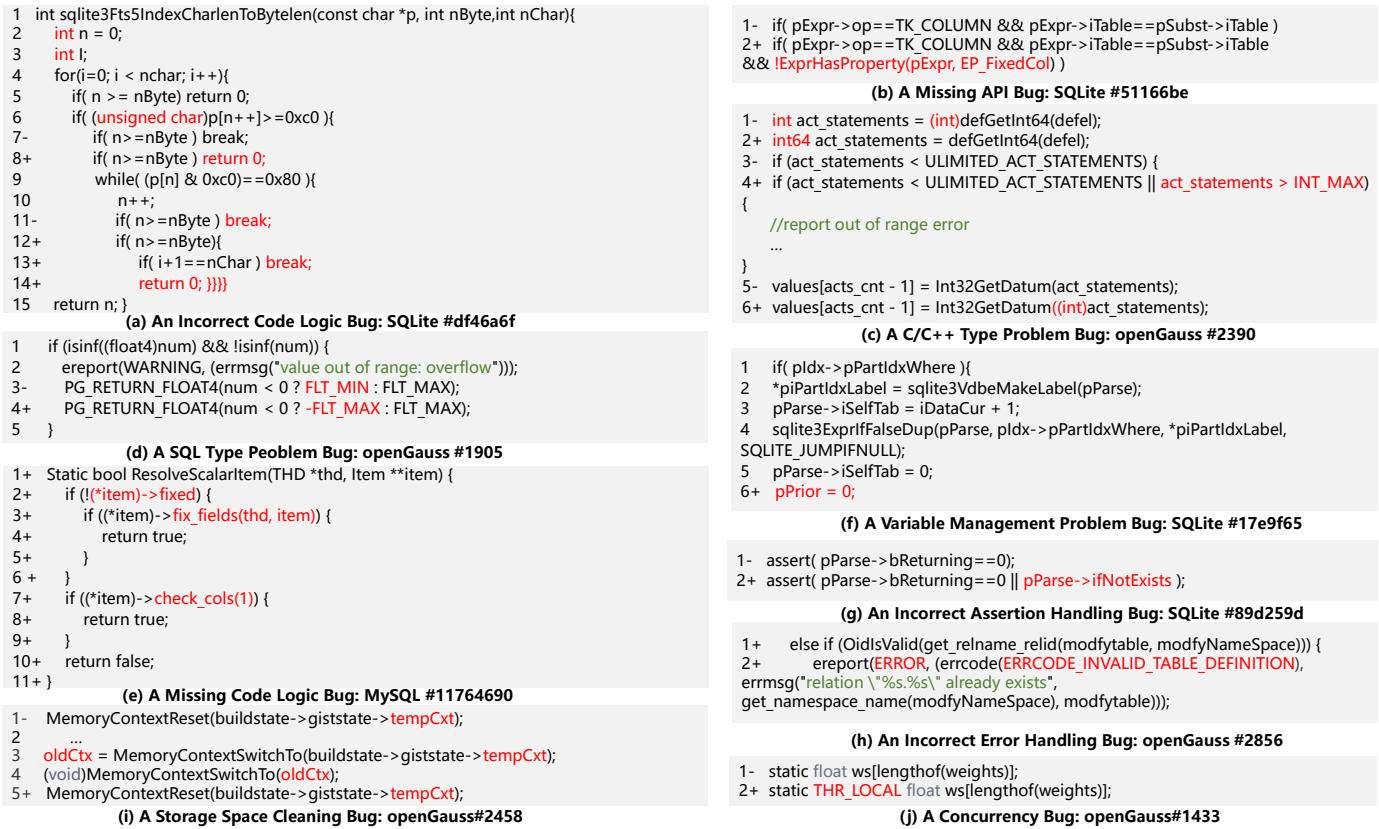


Fig. 4: Examples of Bugs of Different Root Causes

- Database maintenance: bugs related to database configuration and maintenance, such as system settings, logging and data backups.
- Table structure/column attribute related: Bugs concerning table structure or column attribute settings, such as virtual tables and generated columns.
- Transaction related: Bugs related to programming logic processing transactions.
- Other: Miscellaneous bugs that do not fit into the aforementioned categories.

As depicted in Figure 3, bugs related to RDBMS-specific features are the most frequent, comprising 19.12% of all bugs. Optimization and SQL expression errors also constitute substantial portions, while other functionalities exhibit a more evenly distributed occurrence of bugs. This distribution highlights that incorrect code logic is widespread across various RDBMS functionalities.

Figure 4 (a) presents an incorrect code logic bug, originating from SQLite's FTS5 module, a SQLite-specific text indexing subsystem. The bug leads to an assertion failure in debug mode. This function computes the number of bytes required to represent the first `nChar` UTF-8 characters in the buffer pointed to by `p`. The total length of the buffer is specified by `nByte`. Two counters, `i` tracks the number of characters processed, and `n` records the total number of bytes accumulated. Since UTF-8 is a variable-length encoding, the method identifies single-byte (line 6 evaluates to false) and multi-byte characters (line 6 evaluates to true) separately. When a multi-byte UTF-8 character is encountered, it continues reading the subsequent bytes until the

full character has been decoded (lines 9-14).

The logical flaw arises when the input buffer ends in the middle of decoding a multi-byte character. The original code (lines 7 and 11) handles buffer exhaustion using `break`, allowing the loop to terminate normally and return the current byte count (line 15) as if the character had been fully processed. This violates the constraint that every decoded character must consist a complete UTF-8 byte sequence. Consequently, the function returned an incorrect byte count, ultimately triggering an assertion failure.

In the code fix, the developer replaced both `break` statements with `return 0`, ensuring that the function immediately reports an error whenever the buffer ends before a complete character is decoded. Only when the boundary coincides exactly with the last requested character (line 13) can the function terminate normally. This fix restored the correct termination semantics, distinguishing between "buffer exhaustion" and "proper boundary completion". Consequently, the fix ensured that the function's behavior strictly adhered to UTF-8 semantics. This is a representative bug of *Incorrect Code Logic*, where some of the internal logic of the program is incorrectly designed and implemented.

Finding 1: Incorrect code logic, as the most prevalent root cause, accounts for 32.30% of all analyzed bugs. These bugs are widespread across various database functionalities, with RDBMS-specific features, optimizations, and SQL expressions being the most common. Specifically, they represent 19.12%, 14.34%, and 13.94% of all bugs of this root cause, respectively.

TABLE 3: Distribution of API Misuse Bugs

	SQLite	MySQL	openGauss	Sum
M/R API	15	14	24	53
API Args	8	6	11	25
Wrong API	4	3	1	8
API Timing	2	1	4	7

API Misuse is the second common root cause of bugs in RDBMS, as illustrated in Figure 2. There are 93 bugs due to API misuse, with 24 from MySQL, 29 from SQLite, and 40 from openGauss, collectively representing 11.97 % of all studied bugs.

Bugs in this category can be further classified into four subcategories: (1) Missing/Redundant API (M/R API) refers to the cases where an API is either not utilized when necessary or used redundantly; (2) Incorrect API argument (API Args) refers to the cases of passing incorrect parameters to the API; (3) Incorrect API usage refers to the cases of using an incorrect API for the intended purpose; and (4) Incorrect API usage timing (API Timing) refers to the cases of calling the APIs too early or late in the program. Table 3 details the prevalence of these subcategories, indicating that Missing/Redundant API (56.99 %) and Incorrect API Argument (26.88 %) are more frequently encountered than Wrong API (8.60 %) and Wrong API Usage Timing (7.53 %). This distribution aligns with findings from prior studies on API misuse [41], [42].

Figure 4 (b) showcases a specific instance of a Missing API bug that led to an assertion failure. The bug appeared during the push-down optimization process, where the code fails to check for the special cases of constants when deciding whether to push down a subquery, potentially triggering assertion failures. The fix invoked the `ExprHasProperty` API to properly address this special case.

Finding 2: API misuse is the second most frequent root cause, representing 11.97 % of all studied bugs. Missing/Redundant API is the most frequently occurring subcategory in API misuse, accounting for 56.99 % of all API Misuse bugs.

Type problems emerge as the third most prevalent root cause, accounting for 92 bugs across the bugs analyzed (27 from MySQL, 39 from SQLite, and 26 from openGauss), which represents 11.84 % of all identified bugs. Among these, 9.78 % are related to C/C++ data types, while 90.22 % pertain to SQL data types. Figure 4 (c) illustrates a specific instance of a C type-related bug in openGauss. In this scenario, the variable `act_statements`, which denotes the maximum concurrency allowed by the resource pool, was initially declared as a 32-bit integer (`int`). It erroneously receives a value from a 64-bit integer (`int64`), forcing a conversion that may result in a very small `int32` value due to exceeding the range that an `int32` can represent. This conversion leads to the assignment of an incorrect value to `values[act_cnt-1]` on line 5, resulting in inconsistent results. The resolution involved redeclaring `act_statements` as a 64-bit integer (line 2) and adding a safeguard: the variable's value is checked

against `INT_MAX` before any conversion (line 4). If the value exceeds `INT_MAX`, an error is reported, ensuring that any subsequent conversion to a 32-bit integer (line 6) maintains data integrity.

Figure 4 (d) shows a bug caused by incorrect SQL type conversion, resulting in inconsistent results. This bug is in the SQL type conversion function, which converts the `float8` type (double precision) to the `float4` type (single precision). When a `float8` value that exceeds the range that `float4` can accommodate, the value should logically be converted to the smallest value supported by `float4`. However, the buggy code wrongly set the lower bound of a `float4` number to `FLT_MIN`, and the fix changes it to the correct number of `-FLT_MIN`. Our analysis reveals that SQL type problems constitute over 10% of all studied bugs, with 69% of these bugs exhibiting the symptom of result inconsistency. These are silent bugs that are difficult to detect. Current testing approaches for RDBMSs typically overlook SQL type features, resulting in these bugs being under-tested [23]. Consequently, targeted efforts are necessary to address bugs related to incorrect SQL types effectively.

Finding 3: Among all bugs attributed to type problems, a significant majority (90.22 %) are related to incorrect handling of SQL data types, accounting for more than 10% of all studied bugs. Incorrect SQL type bugs are overlooked by current testing methods and targeted efforts are necessary to address these bugs.

Missing code logic accounts for 91 identified bugs. Figure 4 (e) illustrates a bug related to missing code logic. In this case, a function is intended to identify whether a column represents a scalar value, returning `false` if it is scalar and `true` otherwise. This code logic, which is critical in operations such as `CREATE EVENT` and `ALTER EVENT`, where misidentifying an item as scalar could have adverse effects, was missing in the original code.

Variable management issues contribute to 60 bugs, accounting for 7.72 % of all identified bugs. These include 33 instances of incorrect variable assignments, 10 errors in variable computations, 9 initialization errors, and 8 cases of variable overflow. Figure 4 (f) highlights an incorrect variable assignment issue. The code from lines 1 to 5 is designed to test each record of a partial index to evaluate its usability. Here, `pPrior`, a pointer to the column values of an index record, is involved. After completing the tests, `pPrior` should be reset to 0 to prevent subsequent erroneous operations. However, the original code failed to reset `pPrior`, potentially leading to incorrect results when later code checks this pointer. The issue was fixed by adding code to correctly reset `pPrior` to 0.

Among the 54 documented cases of incorrect Error/Assertion handling bugs, 70.37 % are related to error handling and 29.63 % pertain to assertion handling errors. Figure 4 (g) illustrates a bug caused by incorrect assertion handling, where a necessary check for the `ifNotExists` parameter in the `pParse` pointer was omitted, erroneously triggering an assertion failure. Figure 4 (h) displays an error handling issue encountered during table renaming. The original code lacked a duplication check, leading to abnormal error messages subsequently. The resolution involved

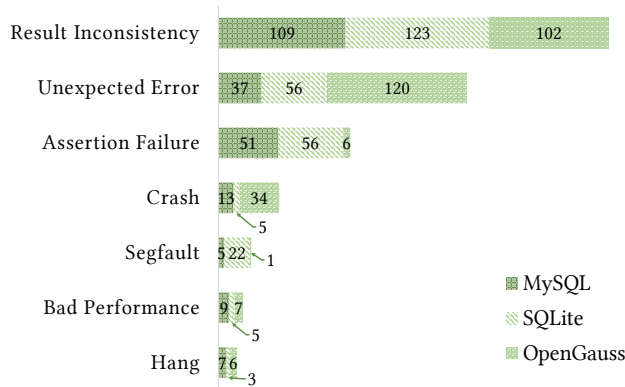


Fig. 5: Bug Distribution by Symptoms

introducing a new check to determine if the table name was already in use, thus preventing an `INVALID TABLE DEFINITION` error if a duplicate was found.

Regarding the 45 bugs associated with DBMS storage management, 44.44 % are due to incorrect storage space allocation, 40.00 % relate to improper storage space cleanup, while the remaining 15.56 % are related to storage space initialization. Figure 4 (i) depicts a bug arising from faulty storage cleanup. Initially, the pointer `oldCtx` was set to reference the memory location previously pointed to by `tempCtx`, which had been prematurely cleared (line 1). This mismanagement led to potential memory leaks when `oldCtx` was accessed (line 4). The correction involved adjusting the cleanup of `tempCtx` to occur after its final use (line 5), ensuring proper memory management.

Bugs related to Concurrency, Null Pointer Problems, Incompatibility, Misconfiguration and Typos are relatively rare, accounting for 5.02 %, 2.45 %, 2.32 %, 1.42 %, and 0.51 % of all bugs, respectively. An example of a concurrency-related bug is shown in Figure 4 (j). Initially, the variable `ws` was declared as a static variable within a function, inadvertently making it shared across all threads executing that function and leading to writing conflicts. Recognizing that `ws` did not need to be shared, the solution was to apply the `THR_LOCAL` attribute to the variable, ensuring it is thread-local and thereby eliminating the conflicts.

3.2 RQ2: Bug Symptoms

Figure 5 displays the distribution of RDBMS bugs by symptoms. Result inconsistency emerges as the predominant symptom, representing 42.99 % of all bugs. Each of the three studied RDBMS systems reports over 100 bugs categorized under this symptom. Result inconsistency typically does not produce explicit error messages or lead to system crashes, making these bugs particularly challenging to detect. Common causes of result inconsistencies include incorrect code logic (34.73 %), type problems (18.56 %), and API misuse (14.67 %). A detailed correlation analysis between these root causes and symptoms will be presented in Section 3.4.

Finding 4: Result inconsistency is the most frequent symptom encountered in RDBMS bugs, accounting for 42.99 % of all cases. These bugs often manifest silently, without system crashes or explicit error messages, necessitating increased effort for detection.

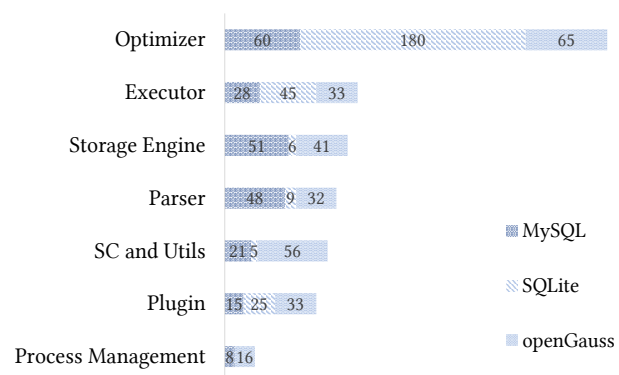


Fig. 6: Bug Distribution by Modules

Unexpected error messages are the second most common symptom of bugs, accounting for 27.41 % of all identified issues. These bugs typically produce explicit error messages; however, the messages are often misleading, leading users and developers to incorrect conclusions. Among the 213 bugs associated with unexpected errors, 69 are due to incorrect code logic, 26 stem from API misuse, and 19 are related to type problems. The misleading nature of the error messages makes these bugs particularly challenging to locate and resolve.

Assertion failures, which also produce explicit error messages, are comparatively easier to identify. The distribution of assertion failures across RDBMS systems reveals more incidents in MySQL and SQLite compared to openGauss. This disparity could suggest that openGauss may have undergone less rigorous testing or have conducted thoroughly internal testing before code release.

Finding 5: Unexpected Error and Assertion Failure are the second and third most prevalent symptoms in RDBMS, accounting for 27.41 % and 14.54 % of all bugs, respectively.

System crashes, including crashes and segmentation faults, represent a critical category of bugs. In our study, we identified a total of 52 crashes and 28 segmentation faults. Segmentation faults typically occur when a program attempts to access a restricted memory area. Such bugs, often caused by null pointer problems or incorrect storage management, are usually highly critical, indicating severe bugs or potential vulnerabilities within the RDBMS. Among the systems we studied, openGauss and SQLite exhibited the higher number of crashes and segmentation faults than MySQL. This suggests varying levels of robustness and error handling across different RDBMS implementations.

Bad Performance and Hang are uncommon in the analyzed bugs, accounting for 2.70 % and 2.06 % of all bugs.

Finding 6: Bugs that cause system crashes or segmentation faults account for 10.30 % of all studied bugs. Although these issues are not frequent, they are usually critical and tend to lead to severe outcomes, necessitating increased attention. In contrast, system hangs and poor performance are uncommon, comprising 2.70 % and 2.06 % of all bugs, respectively.

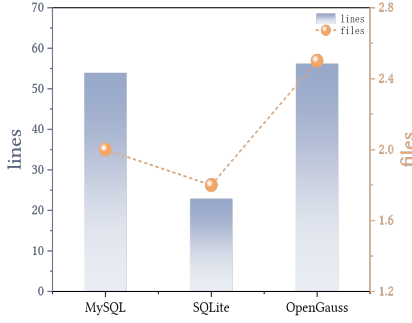


Fig. 7: Fix Lines and Influenced Files

TABLE 4: Bug Severity Levels of Different RDBMS

level	MySQL	SQLite	openGauss
Critical	S1	immediate	severe
Major	S2	high	primary, secondary
Minor	S3, S5, S6	medium, low	unimportant, unspecified

3.3 RQ3: Bugs Distribution in Modules

We analyzed the bug distribution in modules, as well as the bug fix efforts and bug severity levels.

Figure 6 illustrates the distribution of bugs by modules within RDBMSs. The optimizer module harbors the most bugs—a total of 305 bugs, representing 39.25 % of all issues. This includes 60 from MySQL, 180 from SQLite, and 65 from openGauss. The optimizer, as the largest and most complex module, comprises various optimization algorithms, making it particularly prone to errors. SQLite, being a lightweight embedded RDBMS that demands high performance with a smaller code size (only 8.77% of MySQL's), exhibits a higher number of bugs in this module.

The Executor module contains 106 bugs, accounting for 13.64 % of the total. It plays a critical role in executing SQL queries after they are processed by the parser and optimizer, handling tasks such as executing query plans, managing transactions, and controlling concurrency. The complex logic and extensive codebase of the Executor module make it susceptible to bugs.

The Storage Engine, Parser, and Shared Utils modules have 98, 89, and 82 bugs, respectively. SQLite shows the fewest bugs in these modules, while openGauss has the highest. Surprisingly, MySQL's Parser module also exhibits many bugs, despite its maturity and the general perception of its well-developed parser.

Plugin-related bugs primarily occur in specific extensions, e.g., the FTS extension in SQLite, group replication in MySQL, and the dolphin extension mode in openGauss. Process Management, which handles thread management and is typically related to concurrency, contains the fewest bugs, with only 24. Notably, SQLite, which operates as a single-threaded DBMS, has only one reported bug in this module—a performance issue related to using coroutines to avoid materializing views. MySQL and openGauss have 8 and 16 bugs in this module, respectively.

Finding 7: The optimizer, executor, and storage engine modules are the top three bug-prone modules in RDBMS, containing 39.25 %, 13.64 %, and 12.61 % of all studied bugs, respectively.

TABLE 5: Bug Distribution on Severity Levels

security	MySQL	SQLite	openGauss
Critical	7	176	9
Major	24	10	184
Minor	38	84	83
Total	69	270	276

Figure 7 shows the average number of fixed lines and influenced files. On average, fixing a bug requires modifying 53.95 lines across 2 files. openGauss exhibits the highest average with 56.19 lines and 2.50 files per fix, while SQLite requires the least effort with only 22.87 lines and 1.80 files. MySQL, on the other hand, shows a higher average in lines (53.95) but a lower average in files (2.0). This variation in bug fixing effort could be attributed to differences in code organization: MySQL's code files are organized based on RDBMS functionalities, whereas openGauss's files are organized by modules, with each functionality potentially spanning multiple functions and files. Consequently, bug fixes in openGauss often involve more files compared to MySQL and SQLite.

Each development team for the studied RDBMSs assigns their own severity levels to bugs. MySQL categorizes issues using seven labels: S1 (Critical), S2 (Serious), S3 (Non-critical), S4 (Feature request), S5 (Performance), S6 (Debug Builds), and S7 (Test cases), with S1, S2, S3 and S6 specifically relating to bug severity. SQLite employs severity levels of immediate, high, medium, and low, while openGauss uses labels of severe, primary, secondary, unimportant, and unspecified. To facilitate a comparative analysis across these varying priority systems, we mapped the severity levels from each RDBMS into three standard categories: critical, major, and minor, as detailed in Table 4.

Table 5 presents the distribution of bug severities for each RDBMS. Notably, only a small percent of MySQL bugs are assigned severity labels. SQLite has the highest percentage of bugs (65.19%) classified as critical and the lowest classified as major. In contrast, most of the bugs in openGauss are categorized as major. This indicates that different RDBMSs not only have their own labeling criteria but also exhibit distinct distributions of severity.

Finding 8: On average, each bug fix involves modifications to 53.95 lines across 2 files. There is notable variation in bug severity labeling criteria and distributions across different RDBMSs.

3.4 RQ4: Correlations

We also analyzed the correlations between the bug root causes and bug symptoms, as well as bug root causes and bug severity levels. In particular, for bugs of each root cause, we calculate their normalized frequencies on each bug symptom with formula (1). N_{rc} is the number of bugs with root cause rc , N_{rc}^{sym} is the number of bugs with root cause rc and showing symptom sym , F_{rc}^{sym} is percentage of bugs showing symptom sym in all bugs with root cause rc . Similarly, we use formula (2) to calculated normalized frequencies on each bug severity level. N_{rc}^{ser} is the number of bugs with root cause rc and is labeled severity level ser ,

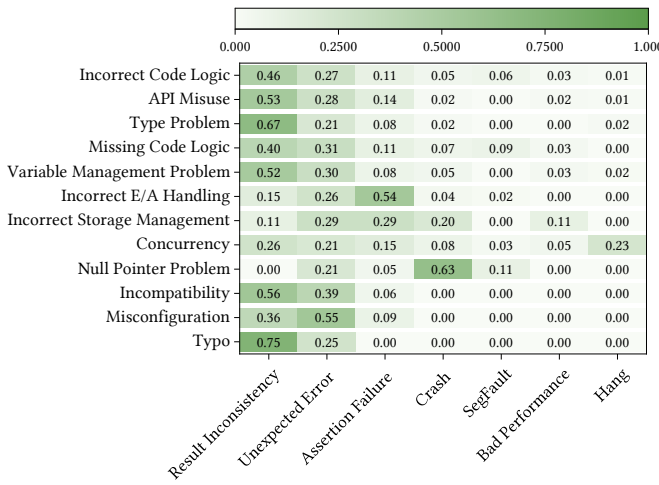


Fig. 8: Relation between Symptoms and Root Causes

F_{rc}^{ser} is percentage of bugs labeled severity level ser in all bugs with root cause rc .

$$F_{rc}^{sym} = \frac{N_{rc}^{sym}}{N_{rc}} \quad (1)$$

$$F_{rc}^{ser} = \frac{N_{rc}^{ser}}{N_{rc}} \quad (2)$$

Root Causes and Symptoms Figure 8 illustrates the co-occurrence relationship between root causes and bug symptoms. We can observe that result inconsistency prominently influences the overall bug distribution. On five root causes, i.e., API Misuse, Type Problem, Variable Management Problem, Incompatibility and Typo, there are more than 50% of bugs exhibiting this symptom. On the other three root causes, i.e., Incorrect Code Logic, Missing Code Logic, Misconfiguration, there are over 30% of their symptoms exhibiting result inconsistencies. These result inconsistencies are particularly challenging to detect as they manifest silently without error messages, warranting a thorough investigation into such bugs. Unexpected errors also appear across all types of root causes, indicating a widespread but distinct symptom category. Specific correlations between root causes and symptoms are evident; for example, 54% of bugs stemming from incorrect error/assertion handling manifest as assertion failures, while 63% of bugs caused by null pointer problems result in crashes. Understanding these correlations can significantly aid in analyzing the root causes of bugs based on their symptoms.

Root Causes and Severity Level Figure 9 displays the relationship between root causes of bugs and their severity levels. It is evident that bugs stemming from incorrect code logic, type problems, variable management and incorrect storage management are more often labeled severity. In contrast, bugs related to concurrency, incompatibility, null pointer problems, API misuse, incorrect error/assertion handling and missing code logic are less often categorized as critical and are more frequently labeled as major.

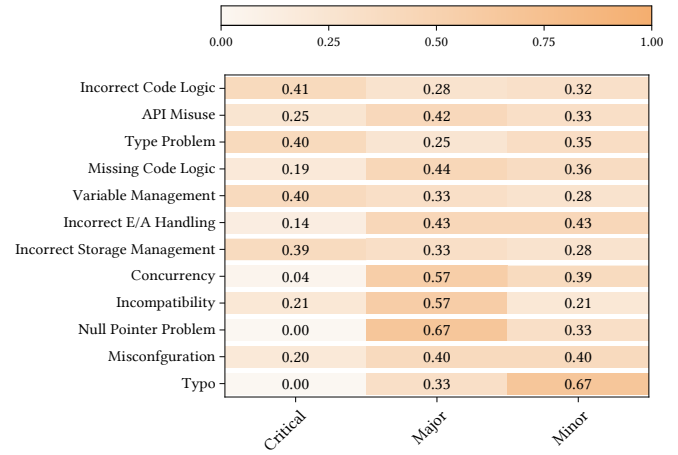


Fig. 9: Relation between Root Causes and Severity Level

TABLE 6: The Spearman Correlation of RDBMSs

RDBMS	Symptoms	Root Causes
SQLite & MySQL	0.99	0.88
SQLite & openGauss	0.70	0.67
MySQL & openGauss	0.70	0.70

Finding 9: Result inconsistency is a symptom potentially caused by a wide range of root causes (11 out of 12 analyzed), often manifesting silently and complicating detection. This complexity necessitates targeted efforts to effectively address these issues, given their subtle yet impactful nature.

To further measure the correlations of the bugs between different RDBMSs, we report the Spearman's correlation result in Table 6. The results show that SQLite and MySQL have a high correlation on both symptoms and root causes, while their correlation with openGauss is relatively low. openGauss is based on PostgreSQL, having a different architecture and different focused functionalities from MySQL and SQLite. Moreover, openGauss is under active developing, making the bugs in openGauss have some offset in the root cause compared to SQLite and MySQL.

Finding 10: SQLite and MySQL have a high correlation on both bug symptoms and root causes, and their correlations with openGauss is relatively low.

3.5 Bug distributions across studied RDBMSs

We further analyzed bug distributions across three database systems from three perspectives, shown in Figure 10.

In terms of bug symptoms, MySQL and SQLite exhibit similar distributions, whereas openGauss shows a significantly higher proportion of Unexpected Errors. This can be attributed to its ongoing active development phase, during which system instability—particularly at the parsing and execution layers—is more likely. Additionally, openGauss has fewer assertion failures but a higher number of crashes, suggesting that it lacks sufficient assertion coverage to detect and halt program failures gracefully.

Regarding root causes, all three systems share broadly similar patterns. However, Incorrect Code Logic errors are

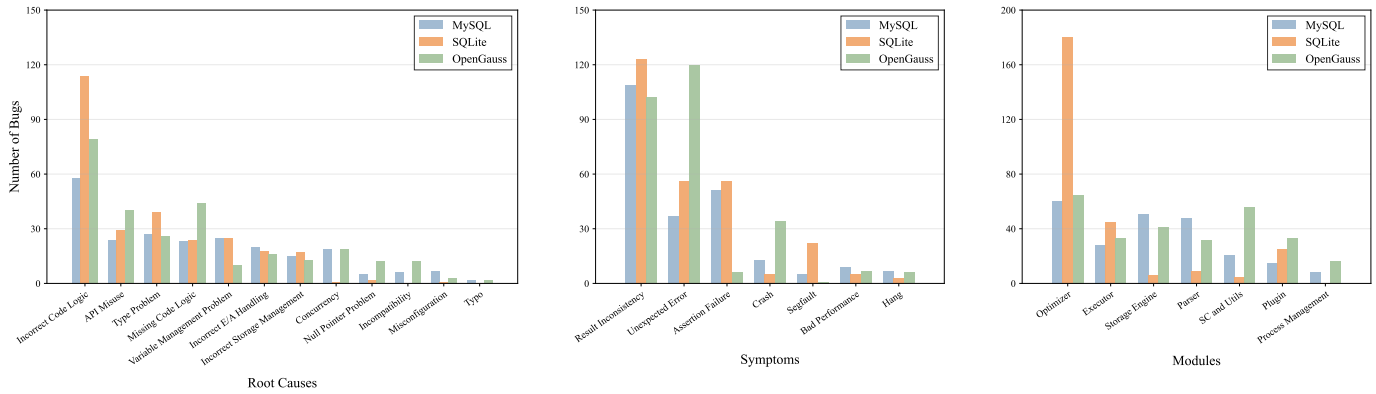


Fig. 10: Bug Distributions across Different RDBMSs

noticeably more frequent in SQLite. This is primarily due to its heavy reliance on complex optimization strategies, which lead to a disproportionately large and intricate optimizer codebase—thereby increasing the likelihood of logic-related bugs. openGauss shows a higher incidence of Missing Code Logic, consistent with its active developing status and relative immaturity in handling corner cases.

For affected modules, SQLite reports the highest number of bugs in the optimizer. As an embedded database, SQLite features relatively lightweight execution and storage engines but a more substantial optimizer component. This architectural focus explains why its bugs are concentrated in the optimization module.

3.6 Bug-triggering SQLs

We further analyzed the characteristics of the bug-triggering test cases—i.e., the SQL statements—focusing specifically on those that are not supported by existing test case generation tools such as SQLancer and SQLSmith. Across the three studied DBMSs, we identified 146 such cases that current tools cannot generate, including 46 in MySQL, 79 in OpenGauss, and 21 in SQLite. As we will discuss, extending current tools to support these features is non-trivial due to the complexity and specificity of the required SQL constructs.

Partition-related bugs are prevalent across all three DBMSs, with a total of 61 cases identified. These bugs often stem from incorrect partition pruning, invalid comparisons involving partition keys, or unsupported operations on partitioned tables. Compared to flat table structures, partitioned tables introduce additional layers of metadata and planning logic, which can significantly affect query behavior and optimization strategies. However, existing test generation tools such as SQLancer and SQLSmith do not support partition-related syntax at all, making it fundamentally impossible for them to generate test cases involving partition logic. This limitation highlights a critical blind spot in current automated testing techniques and underscores the need for extending support for partitioned table constructs in future testing frameworks.

There are 19 bug-triggering test cases involving stored procedure/function and PLSQL statements. Constructing relevant test cases involves multi-statement routines, parameter passing, and internal control flow such as IF and WHILE. These are non-trivial to generate due to their nested

TABLE 7: Feature Occurrences in Studied RDBMSs

	MySQL	SQLite	OpenGauss	Total
Partition	13	10	38	61
Procedure	21	0	20	41
DynamicSQL	16	0	11	27
UserMgmt	7	0	17	24
Event/Trigger	5	11	6	22
Total	62	21	92	175

structure and stateful behavior, making it harder to ensure the semantic correctness of the generated SQLs.

Bug-triggering test cases involve dynamic SQLs, e.g., PREPARE, EXECUTE, and DEALLOCATE PREPARE, involve runtime construction and execution of statements. They require testing both SQL parsing and execution layers, and are prone to injection, type mismatch, and scope issues. There are 27 bugs related to this feature.

There are 24 bug-triggering test cases involving user management features (e.g., in CREATE USER, GRANT, REVOKE) and are tied to access control and privilege settings. These bugs often manifest only under specific combinations of roles, permissions, and isolation levels, which are beyond the expressiveness of current SQL generators.

There are 3 bug-triggering test cases involve database events, such as triggers, which involve scheduled or automatically triggered actions. Bugs can be triggered only under specific cascading update or delete conditions. Designing test cases requires explicitly linking triggering tables, timing (BEFORE/AFTER), and actions—conditions that are rarely satisfied through random testing.

Table 7 presents the occurrences of rare SQL features in the bug-triggering test cases across MySQL, openGauss, and SQLite. Since a single bug-triggering test case may involve multiple features, the total feature occurrence count Table 7 exceeds the number of unique bugs triggered. The results highlight that there are 18.79% of the studied bugs with those rarely supported SQL features and thus cannot be triggered by existing testing tools. There are also 14.93% of studied bugs involve externally setting the database states to trigger the bugs. These bugs are not detected by existing testing tools.

4 DISCUSSION

In this section, we delve into bugs associated with RDBMS-specific features, discussing the implications derived from our findings. We then discuss threats to validity of our study.

4.1 Implications

Transaction Bugs. Transactions are essential in RDBMS for ensuring the safe and reliable processing of data, critical in sectors like banking and online booking systems. However, there are only 33 transaction-related bugs, constituting 4.25 % of all analyzed bugs, with 15 from MySQL, 5 from SQLite, and 13 from openGauss. This finding aligns with a recent research reporting that transaction bugs are under-tested [23]. Recent works on RDBMS transaction testing [25], [21], [20] offer insights into testing efficacy. Our examination of 33 transaction bugs against these methodologies showed that only 5 could potentially be detected by existing tools. The majority, 28 bugs, with feature complexities like clustering, replication, group replication, log analysis, custom functions, and control flow, which are not currently covered by these testing efforts. Importantly, extending these testing tools to include more keywords in test case generation is insufficient; the primary challenge lies in developing effective oracles that can detect these complex transaction bugs. Moreover, current methods typically involve intensive processes like view reconstruction or code instrumentation, requiring significant modifications to implementations for incremented features and isolation levels. This underscores the necessity for more effective strategies in identifying and testing transaction-related bugs in RDBMS systems.

Finding 11: Transaction support is common across the studied RDBMSs, yet transaction bugs account for only 4.25 % of all issues. Remarkably, over 84.85% of the transaction bugs we analyzed could not be detected by existing transaction testing approaches. This finding underscores the need for more dedicated efforts towards addressing RDBMS-specific features, i.e., transaction-related bugs.

Test Case Generation. Our analysis of bug-triggering test cases reveals that certain features—PL/SQL constructs, user access control, and specific database parameter settings—play a critical role in exposing bugs in RDBMSs, despite often falling outside the standard SQL scope. Consequently, it is essential for test case generation tools to support these features. However, extending existing syntax-directed tools like SQLancer to incorporate such functionality is non-trivial, as these features often require complex semantic correctness checks. This highlights the need for a new test case generation method that can inherently address semantic validation in a principled and scalable way.

Finding 12: Approximately 33.72% of the studied bugs require complex features—such as database triggers, events, stored procedures, or specific parameter settings—in their test cases to be triggered. However, existing RDBMS testing tools lack support for these advanced features. Generating such test cases under the current syntax-driven paradigm is challenging, as it often involves intricate semantic correctness checks. This underscores the need for a new test case generation paradigm that can natively support and validate complex semantic behaviors.

4.2 Threats to Validity

The primary internal threat stems from the potential bias in the manual labeling and classification of RDBMS bugs. To mitigate this, we adhere to a standard labeling process and have established clear classification criteria. Each bug is independently labeled by at least two authors, and the Cohen's Kappa coefficient is utilized to assess the agreement between labels. In instances of disagreement, a third author independently reviews the bugs, followed by a discussion among all three authors to reach consensus. Secondly, We drew inspiration from existing methodologies in compiler and database bug studies to design our classification criteria, which we refined iteratively as non-conforming bugs emerged during the labeling process. Another threat to validity concerns the selection and collection of bugs for our study. To address this, we systematically collected RDBMS bugs as detailed in Section 2.3. We included only bugs that were closed, had documented fixes in bug reports or merged pull requests, ensuring that the bugs were analyzed and resolved by developers. To enhance the generalizability of our findings, we chose three popular and diverse RDBMSs, representing different architectures and application scenarios. After a meticulous filtering process, we analyzed 777 bugs, balancing the scope of our research with the demands of manual effort.

5 A POOF-OF-CONCEPT TOOL SQLT

To evaluate the practical utility of our findings in enhancing RDBMS testing, we focus on SQL-type bugs, which account for approximately 9% of all studied bugs. We developed SQLT, a generation-based tool specifically designed to address SQL-type issues, enabling targeted testing and validation based on the identified bug patterns.

5.1 Design and Motivation of SQLT

SQLT is an extension of the widely-used database testing tool, SQLancer [12]. Based on our observation that many SQL-type bugs are triggered by novel or less-tested data types (e.g., BIT, JSON), while others result from implicit type conversions within functions, we enhanced SQLT by incorporating syntax-aware rules. These rules systematically generate SQL statements designed to either introduce new data types or provoke SQL type conversions, both implicit and explicit, thereby increasing the likelihood of exposing type-related bugs. This makes SQLT particularly adept at testing RDBMS functionalities related to SQL types. Specifically, SQLT extends SQLancer in four major aspects.

Supporting new data types. SQLT extends SQLancer by introducing new data types such as BIT and JSON, which were undersupported by SQLancer. Moreover, SQLT adds support for *openGauss* by implementing a dedicated test case generation module.

Encouraging type conversion. SQLT systematically increases the probability of both *explicit* type conversions (e.g., CAST, CONVERT) and *implicit* conversions by injecting heterogeneous data types into expressions, effectively stressing the type conversion logic of DBMSs. SQLT also integrates a large set of database-specific functions that are prone to type conversion issues, such as STRING_AGG, UNIXEPOCH,

TABLE 8: Bug Detected by SQLT

Bug ID	RDBMS	Version	Status
112912	MySQL	8.0	Confirmed
112914	MySQL	8.0	Confirmed
a655ee159e	SQLite	3.43.0	Confirmed
I8GTHH	openGauss	5.0.0	Fixed
I8ENSF	openGauss	5.0.0	Fixed
I879H0	openGauss	5.0.0	Fixed
I8ENDC	openGauss	5.0.0	Fixed
I8ENLG	openGauss	5.0.0	Duplicate

TABLE 9: Comparison of SQLT with SQLancer for 24h

RDBMS	SQLT	SQLancer
MySQL	4	3
SQLite	1	1

and TIMEDIFF in MySQL/openGauss, and RANDOM, IIF, OCTET_LENGTH, and UNHEX in SQLite. These functions are obtained from our analysis of existing bugs and newly introduced functions in recent DBMS releases.

Given that bugs related to SQL types often result in result inconsistencies—silent bugs without explicit error messages—we incorporated two existing metamorphic testing approaches, NOREC [14] and TLP [19], as oracles within SQLT. SQLT is publicly available and can be accessed at <https://github.com/sqlttest/SQLT>.

5.2 Evaluation of SQLT

Our experiments were conducted on an Ubuntu 16.04 server equipped with 500GB of memory and an Intel(R) Xeon(R) Platinum 8260 CPU @ 2.30GHz. We applied SQLT to SQLite versions 3.43.0, MySQL 8.0.34, and openGauss 5.0.0 over a two-week period.

SQLT successfully detected 8 bugs, all of which were confirmed by developers and 4 have been fixed, as detailed in Table 8. Notably, 2 bugs in MySQL were related to type conversion, 1 bug in SQLite was associated with JSON5 SQL type support, and in openGauss, 2 bugs were due to SQL types, with the remaining 3 stemming from incorrect error handling, null pointer errors, and issues with the full join keyword, they were uncovered due to our extended support for openGauss. These findings, particularly the detection of 5 type-related bugs within a short timeframe, validate our initial hypothesis and underscore the need for more focused efforts on SQL type-specific issues in RDBMS testing.

We've also compared SQLT with SQLancer on the latest version of MySQL and SQLite (MySQL 8.0.34 and SQLite 3.43.0)³ for 24 hours and adopt NoREC [14] as the default oracle to detect bugs. SQLT detects 6 bugs and SQLancer detects 4 bugs. Among these results, SQLT uncovered 1 additional bug in MySQL that were *not detected* by SQLancer. The bug stems from **type-related issues** involving implicit conversions and edge-case expressions on the column of BIT type. Notably, SQLancer does not actively avoid type mismatches; rather, its test generation lacks sufficient bias toward diverse data types and fails to synthesize high-risk constructs like nested casts or ambiguous comparisons. SQLT addresses these gaps by increasing the likelihood of

type coercion chains, introducing under-tested types such as BIT and JSON, and embedding conversion-prone expressions in critical control points.

6 RELATED WORK

Empirical Study on RDBMS Bugs There are two RDBMS bug studies [22], [23] in the literature. Fonseca et al. conduct an early study on 80 RDBMS concurrency bugs [22], discussed the impact of the concurrency bugs and the latent concurrency bugs distribution. Cui et al. studied 140 RDBMS transaction bugs [23], including statistical information such as the average amount of data used in a table, the length of code fixes, etc. They also pointed out that the majority of transaction bugs are silent, which is consistent with our findings on RDBMS bugs in general. Both studies focus on a special type of RDBMS bugs, and the number of bugs they studied are small. Our work focus on a broader range of bugs in RDBMS and we studied a total of 777 bugs, making our study general and comprehensive.

RDBMS Testing SQLSmith [11] is the first generation-based testing approach for RDBMS, and it report bugs by examining explicit error messages and crashes. Rigger et al. [12], [14], [19] proposed SQLancer, which is one of the most popular generation-based RDBMS testing approach, and it incorporates three metamorphic methods, namely PQS [12], NoREC [14], and TLP [19], as oracles to detect logical bugs. These testing approaches rely on syntax-directed SQL generation, which lacks support for the complex features analyzed in Section 3.6 and cannot be easily extended to accommodate them. SQUIRREL [13], SQLRight [15] and GRIFFIN [16] are mutation-based testing approaches. SQUIRREL [13] considers both language validity and coverage feedback to test RDBMSs. SQLRight [15] conducts coverage-guided mutation, with semantic validity checkings. GRIFFIN [16] reshuffles statements based on dependencies between metadata and statements in the metadata graph. These testing approaches rely on fuzzing techniques, generating test cases by mutating existing seed inputs. As a result, the quality of the seed test cases plays a crucial role in effectively triggering bugs. While seed cases involving the complex features discussed in Section 3.6 can be partially sourced from open-source projects, ensuring semantic correctness during the mutation process remains a non-trivial challenge. There is also some work dedicated to detecting less common types of bugs in DBMS. AMOEBA [17] and APOLLO [18] test RDBMS performance bugs. Troc [20] and TxCheck [21] are metamorphic testing approaches focusing on detect transactional bugs in RDBMSs. Most of the testing approaches are general, with no focus on RDBMS specific features, like SQL types and transactions. The only three transaction testing approaches have limited capabilities due to the restricted oracles and heavy programming barriers.

7 CONCLUSION

In this paper, we present the first comprehensive study of bugs in Relational Database Management Systems (RDBMS), with the goal of deepening our understanding of RDBMS bugs and improving the RDBMS development,

3. We did not run openGauss as SQLancer does not support openGauss.

testing and maintenance. We meticulously analyzed 777 bugs from three open-source RDBMSs—SQLite, MySQL, and openGauss—covering four key aspects: root causes, bug symptoms, bug fix modules, and the correlations among them. From this analysis, we identified 12 major findings. Building on these insights, we developed a generation-based RDBMS testing tool, SQLT. Within a span of two weeks, SQLT successfully detected 8 significant bugs across the three RDBMSs we studied. This achievement underscores the potential of targeted testing tools that are informed by a detailed understanding of common bug characteristics in RDBMS environments.

ACKNOWLEDGEMENT

This work is supported by the National Natural Science Foundation of China under Grant Nos.6247242, 62322208 and Ant Group through CCF-Ant Research Fund No. CCF-AFSG RF20240103.

REFERENCES

- [1] MySQL, <https://www.mysql.com/>. (visited in December 2023).
- [2] SQLite, <https://sqlite.com/>. (visited in December 2023).
- [3] PostgreSQL, Dec 2023, <https://www.postgresql.org/>. (visited in December 2023).
- [4] OpenGauss, <https://opengauss.org/>. (visited in December 2023).
- [5] WelcomeNext, <https://welcomenext.com/>. (visited in June 2024).
- [6] guard.me, <https://www.guard.me/>. (visited in June 2024).
- [7] ebay, <http://www.ebay.com/>. (visited in June 2024).
- [8] github, <http://www.github.com/>. (visited in June 2024).
- [9] youtube, <http://www.youtube.com/>. (visited in June 2024).
- [10] J. M. Hellerstein, M. Stonebraker, J. Hamilton *et al.*, "Architecture of a database system," *Foundations and Trends® in Databases*, vol. 1, no. 2, pp. 141–259, 2007.
- [11] SQLsmith, <https://github.com/anse1/sqlsmith>. (visited in December 2023).
- [12] M. Rigger and Z. Su, "Testing database engines via pivoted query synthesis," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 667–682.
- [13] R. Zhong, Y. Chen, H. Hu, H. Zhang, W. Lee, and D. Wu, "Squirrel: Testing database management systems with language validity and coverage feedback," in *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, 2020, pp. 955–970.
- [14] M. Rigger and Z. Su, "Detecting optimization bugs in database engines via non-optimizing reference engine construction," in *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2020, pp. 1140–1152.
- [15] Y. Liang, S. Liu, and H. Hu, "Detecting logical bugs of dbms with coverage-based guidance," in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 4309–4326.
- [16] J. Fu, J. Liang, Z. Wu, M. Wang, and Y. Jiang, "Griffin: Grammar-free dbms fuzzing," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
- [17] X. Liu, Q. Zhou, J. Arulraj, and A. Orso, "Automatic detection of performance bugs in database systems using equivalent queries," in *Proceedings of the 44th International Conference on Software Engineering*, 2022, pp. 225–236.
- [18] J. Jung, H. Hu, J. Arulraj, T. Kim, and W. Kang, "Apollo: Automatic detection and diagnosis of performance regressions in database systems," *Proceedings of the VLDB Endowment*, vol. 13, no. 1, pp. 57–70, 2019.
- [19] M. Rigger and Z. Su, "Finding bugs in database systems via query partitioning," *Proceedings of the ACM on Programming Languages*, vol. 4, no. OOPSLA, pp. 1–30, 2020.
- [20] W. Dou, Z. Cui, Q. Dai, J. Song, D. Wang, Y. Gao, W. Wang, J. Wei, L. Chen, H. Wang *et al.*, "Detecting isolation bugs via transaction oracle construction," in *Proceedings of International Conference on Software Engineering (ICSE)*, 2023.
- [21] Z.-M. Jiang, S. Liu, M. Rigger, and Z. Su, "Detecting transactional bugs in database engines via graph-based oracle construction," in *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*, 2023.
- [22] P. Fonseca, C. Li, V. Singhal, and R. Rodrigues, "A study of the internal and external effects of concurrency bugs," in *2010 IEEE/IFIP International Conference on Dependable Systems & Networks (DSN)*. IEEE, 2010, pp. 221–230.
- [23] Z. Cui, W. Dou, Y. Gao, D. Wang, J. Song, Y. Zheng, T. Wang, R. Yang, K. Xu, Y. Hu *et al.*, "Understanding transaction bugs in database systems," in *2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*. IEEE Computer Society, 2024, pp. 947–947.
- [24] db-engines ranking, <https://db-engines.com/en/ranking>. (visited in Aug 2024).
- [25] Z. Cui, W. Dou, Q. Dai, J. Song, W. Wang, J. Wei, and D. Ye, "Differentially testing database transactions for fun and profit," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–12.
- [26] J. Ba and M. Rigger, "Cert: Finding performance issues in database systems through the lens of cardinality estimation," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 2024, pp. 1–13.
- [27] A. Bovenzi, D. Cotroneo, R. Pietrantuono, and S. Russo, "On the aging effects due to concurrency bugs: A case study on mysql," in *2012 IEEE 23rd International Symposium on Software Reliability Engineering*. IEEE, 2012, pp. 211–220.
- [28] Q. Shen, H. Ma, J. Chen, Y. Tian, S.-C. Cheung, and X. Chen, "A comprehensive study of deep learning compiler bugs," in *Proceedings of the 29th ACM Joint meeting on european software engineering conference and symposium on the foundations of software engineering*, 2021, pp. 968–980.
- [29] J. Chen, Y. Liang, Q. Shen, J. Jiang, and S. Li, "Toward understanding deep learning framework bugs," *ACM Transactions on Software Engineering and Methodology*, vol. 32, no. 6, pp. 1–31, 2023.
- [30] J. Garcia, Y. Feng, J. Shen, S. Almanee, Y. Xia, Chen, and Q. Alfred, "A comprehensive study of autonomous vehicle bugs," in *Proceedings of the ACM/IEEE 42nd international conference on software engineering*, 2020, pp. 385–396.
- [31] H. Ma, W. Zhang, Q. Shen, Y. Tian, J. Chen, and S.-C. Cheung, "Towards understanding the bugs in solidity compiler," *arXiv preprint arXiv:2407.05981*, 2024.
- [32] SQLite Bugs, <https://sqlite.org/src/rptview/1>. (visited in December 2023).
- [33] MySQL commits, <https://github.com/mysql/mysql-server/commits/>. (visited in December 2023).
- [34] OpenGauss Bugs, <https://gitee.com/opengauss/opengauss-server/issues>. (visited in December 2023).
- [35] S. M. Vieira, U. Kaymak, and J. M. Sousa, "Cohen's kappa coefficient as a performance measure for feature selection," in *International conference on fuzzy systems*. IEEE, 2010, pp. 1–8.
- [36] C. Sun, V. Le, Q. Zhang, and Z. Su, "Toward understanding compiler bugs in gcc and llvm," in *Proceedings of the 25th international symposium on software testing and analysis*, 2016, pp. 294–305.
- [37] Y. Xiong, M. Xu, T. Su, J. Sun, J. Wang, H. Wen, G. Pu, J. He, and Z. Su, "An empirical study of functional bugs in android apps," in *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2023, pp. 1319–1331.
- [38] MySQL Architecture, <https://dev.mysql.com/doc/refman/8.0/en/pluggable-storage-overview.html>. (visited in December 2023).
- [39] SQLite Architecture, <https://www.sqlite.org/arch.html>. (visited in December 2023).
- [40] OpenGauss Architecture, <https://gitee.com/opengauss/opengauss-server>. (visited in December 2023).
- [41] S. Amann, S. Nadi, H. A. Nguyen, T. N. Nguyen, and M. Mezini, "Mubench: A benchmark for api-misuse detectors," in *Proceedings of the 13th international conference on mining software repositories*, 2016, pp. 464–467.
- [42] S. Amann, H. A. Nguyen, S. Nadi, T. N. Nguyen, and M. Mezini, "A systematic evaluation of static api-misuse detectors," *IEEE Trans. Software Eng.*, vol. 45, no. 12, pp. 1170–1188, 2019. [Online]. Available: <https://doi.org/10.1109/TSE.2018.2827384>